

# Machine Learning on Big Data

## Project Report

### Finding Connected Components in Large-Scale Graphs Using the CCF Algorithm in Apache Spark

---

Based on: *CCF: Fast and Scalable Connected Component Computation in MapReduce*

Kardes et al., inome Inc., Bellevue WA

---

Master M2 — IASD

Université Paris Dauphine–PSL

April 2026

Submitted by: Mahdi Salahshour

## Contents

|          |                                            |           |
|----------|--------------------------------------------|-----------|
| <b>1</b> | <b>Description of the Adopted Solution</b> | <b>2</b>  |
| 1.1      | Problem Statement . . . . .                | 2         |
| 1.2      | The CCF Algorithm . . . . .                | 2         |
| 1.3      | Convergence Criterion . . . . .            | 2         |
| 1.4      | Implementation Strategy . . . . .          | 2         |
| <b>2</b> | <b>Algorithm Design</b>                    | <b>3</b>  |
| 2.1      | CCF-Iterate . . . . .                      | 3         |
| 2.2      | CCF-Dedup . . . . .                        | 4         |
| 2.3      | Full CCF Loop . . . . .                    | 4         |
| 2.4      | Worked Example . . . . .                   | 4         |
| 2.5      | Key Implementation Notes . . . . .         | 5         |
| <b>3</b> | <b>Experimental Analysis</b>               | <b>5</b>  |
| 3.1      | Experimental Setup . . . . .               | 5         |
| 3.2      | Correctness Verification . . . . .         | 6         |
| 3.3      | Scalability Results . . . . .              | 6         |
| 3.4      | Runtime Charts . . . . .                   | 6         |
| 3.5      | Discussion of Scalability . . . . .        | 8         |
| <b>4</b> | <b>Analysis: Strengths and Weaknesses</b>  | <b>9</b>  |
| 4.1      | Strengths of the CCF Algorithm . . . . .   | 9         |
| 4.2      | Weaknesses of the CCF Algorithm . . . . .  | 9         |
| 4.3      | RDD vs DataFrame: Trade-offs . . . . .     | 9         |
| 4.4      | Python vs Scala: Observations . . . . .    | 10        |
| <b>A</b> | <b>Source Code</b>                         | <b>11</b> |
| A.1      | Python Implementations . . . . .           | 11        |
| A.1.1    | ccf_rdd.py . . . . .                       | 11        |
| A.1.2    | ccf_dataframe.py . . . . .                 | 14        |
| A.1.3    | experiments.py . . . . .                   | 17        |
| A.1.4    | graph_utils.py . . . . .                   | 22        |
| A.2      | Scala Implementations . . . . .            | 25        |
| A.2.1    | CCFRdd.scala . . . . .                     | 25        |
| A.2.2    | CCFDataFrame.scala . . . . .               | 28        |
| A.2.3    | Experiments.scala . . . . .                | 31        |
| A.2.4    | GraphUtils.scala . . . . .                 | 34        |

# 1. Description of the Adopted Solution

## 1.1 Problem Statement

Finding connected components in a graph is a common problem in computer science. It has many real-world applications, such as social network analysis, image processing, and record linkage. A connected component is the largest possible group of vertices where you can find a path between any two vertices in that group.

Formally, if we have an undirected graph  $G = (V, E)$ , the goal is to partition  $V$  into disjoint subsets  $C_1, C_2, \dots, C_n$  so that:

$$C_1 \cup C_2 \cup \dots \cup C_n = V, \quad C_i \cap C_j = \emptyset \ (i \neq j),$$

and inside each  $C_i$ , every pair of vertices is connected by a path. At the same time, no path crosses between different subsets, like  $C_i$  and  $C_j$ .

When graphs have billions of nodes and edges, standard sequential algorithms are not practical. To solve this, this project uses a distributed MapReduce-style approach designed for Apache Spark. It follows the algorithm proposed by Kardes et al. [1].

## 1.2 The CCF Algorithm

I use the **Connected Component Finder (CCF)** algorithm introduced in [1]. The main idea is to represent each connected component by its *minimum node identifier*. The algorithm repeatedly passes this minimum ID to all nodes until no further changes can be made.

The whole process uses two linked MapReduce jobs that run over and over until convergence:

1. **CCF-Iterate**: builds adjacency lists and passes the minimum node label to all neighbours.
2. **CCF-Dedup**: removes duplicate pairs created by CCF-Iterate. This lowers the I/O cost for the next step.

## 1.3 Convergence Criterion

During CCF-Iterate, if a node with key  $k$  finds a neighbour with a value  $m < k$ , it produces a *cross-pair*  $(v, m)$  for every other neighbour  $v \neq m$ . A global counter called **NewPair** counts these cross-pairs. If **NewPair** is zero at the end of an iteration, it means no node found a neighbour with a smaller ID. This shows that all components are fully identified, and the algorithm stops.

## 1.4 Implementation Strategy

I implemented CCF in **four distinct versions** within Apache Spark:

**Python RDD**: uses Spark's low-level RDD API with a `LongAccumulator` to track convergence.

**Python DataFrame**: uses the Spark SQL / DataFrame API with a `checkpoint()` call at every iteration to avoid query plan explosion.

**Scala RDD:** same logic as the Python RDD version but compiled to JVM bytecode, eliminating Python-to-JVM serialisation overhead.

**Scala DataFrame:** same logic as the Python DataFrame version, compiled to Scala.

All four implementations produce identical results on the same input graphs, as verified against a reference Union-Find implementation Python and Scala.

## 2. Algorithm Design

### 2.1 CCF-Iterate

Below algorithm presents the CCF-Iterate procedure as described in the original paper and as implemented in this project.

#### Algorithm 1 — CCF-Iterate

```
# --- MAP PHASE ---
# For each edge (a, b), emit two directed pairs to make
# the graph effectively undirected inside the reducer.
def map(a, b):
    emit(a, b)
    emit(b, a)

# --- REDUCE PHASE ---
# key      : a node identifier
# values   : all neighbours of 'key' in the current graph
def reduce(key, values):
    neighbors = list(values)          # materialise iterator
    min_val   = min(neighbors)       # find the minimum neighbour

    # Only act if this node is NOT already the minimum.
    # If min_val >= key, this node is the smallest in its
    # neighbourhood -> nothing to propagate.
    if min_val < key:

        # Type-1 pair: this node adopts the minimum label
        emit(key, min_val)

        # Type-2 pairs: propagate minimum to all OTHER neighbours.
        # Each emission advances the convergence counter (NewPair).
        for v in neighbors:
            if v != min_val:
                NewPair.increment(1) # global convergence counter
                emit(v, min_val)
```

**Why the guard `min_val < key` is essential.** Without this guard, every node would produce cross-pairs in every iteration, even after convergence. Because of this, `NewPair` would never reach zero, which causes an infinite loop. The guard makes sure that only nodes that are *not yet* the minimum of their neighbourhood perform propagation.

## 2.2 CCF-Dedup

### Algorithm 2 — CCF-Dedup

```

# --- MAP PHASE ---
# Use the pair itself as the key so the reducer groups
# duplicates of the same (node, component) together.
def map(key, value):
    emit((key, value), null)

# --- REDUCE PHASE ---
# Each unique (key, value) pair passes through exactly once.
def reduce(key_pair, values):
    emit(key_pair.node, key_pair.component)

```

CCF-Iterate can emit the same pair  $(v, m)$  many times if several nodes share the same neighbour. Without deduplication, the input for the next CCF-Iterate step would be much larger than needed. This would greatly increase I/O and processing time. CCF-Dedup removes these duplicates easily by using a standard grouping operation (in Spark: `distinct()`).

## 2.3 Full CCF Loop

Figure 1 shows the overall structure of the algorithm.

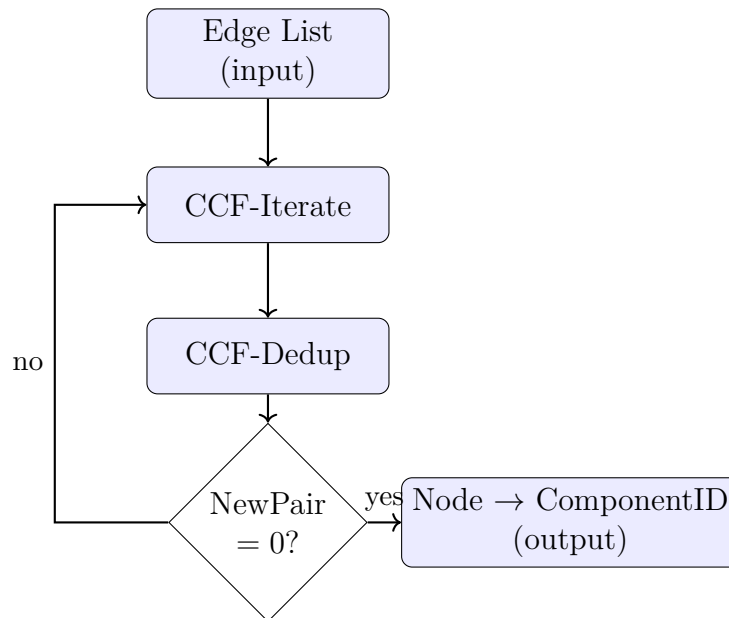


Figure 1: CCF module: iterative chain of CCF-Iterate and CCF-Dedup jobs.

## 2.4 Worked Example

Table 1 traces the algorithm on the six-edge example from Figure 5 of the paper (nodes: A=0, B=1, C=2, D=3, E=4, F=5, G=6, H=7).

Table 1: CCF execution trace on the paper example.

| Iteration | Pairs (after Dedup) | New cross-pairs | Converged? |
|-----------|---------------------|-----------------|------------|
| 1         | 10                  | 4               | No         |
| 2         | 8                   | 9               | No         |
| 3         | 6                   | 4               | No         |
| 4         | 6                   | 0               | Yes        |

After iteration 4 the output maps every node to component 0 (for  $\{A,B,C,D,E\}$ ) and component 5 (for  $\{F,G,H\}$ ), which is exactly the expected result.

## 2.5 Key Implementation Notes

**RDD version — Accumulator pattern.** The `NewPair` counter is a Spark `LongAccumulator`. You must read it *after* a `count()` action forces the DAG to run. If you read an accumulator before any action, it returns old (zero) values. This happens because of Spark’s lazy evaluation model.

**DataFrame version — Checkpointing.** Without explicit checkpointing, each iteration adds another layer of `union` / `groupBy` / `join` / `filter` to Spark’s logical plan tree. After about five iterations, this nested plan uses up all the JVM heap memory when Spark tries to serialise it. Calling `df.checkpoint()` saves the DataFrame to disk and replaces the complex plan with a simple read operation. This keeps memory usage the same across all iterations.

**Complexity.** The algorithm needs between  $d/2 + 1$  and  $d + 1$  iterations, where  $d$  is the diameter of the largest connected component [1]. Each iteration includes one full shuffle (the `groupByKey` / `groupBy` step). Because of this, the total cost scales as  $O(d \cdot |E|)$  distributed operations.

## 3. Experimental Analysis

### 3.1 Experimental Setup

All experiments were run on an Onyxia JupyterHub single-node environment with the following configuration:

**Runtime:** Apache Spark 4.1.1 (Python) / Spark 4.1.1 compiled with Scala 2.13.17 (Scala)

**JVM:** OpenJDK 21.0.10 (64-bit)

**Mode:** `local[*]` (all available cores)

**Graphs:** Random undirected graphs generated with a fixed seed (42) at increasing sizes

**Edge density:** approximately  $3 \times |V|$  edges per graph (average degree  $\approx 6$ )

**Metric:** wall-clock time of the full CCF loop (CCF-Iterate + CCF-Dedup until convergence)

### 3.2 Correctness Verification

Before running scalability experiments, we tested both the RDD and DataFrame versions (in Python and Scala) to make sure they were correct. We used two small graphs for this:

1. The six-edge example from Figure 5 of the paper. This should produce exactly two components in four iterations.
2. A synthetically generated graph with three separate clusters of five nodes each. This should produce exactly three components.

All four implementations passed both tests. Also, the RDD and DataFrame versions produced the exact same pair counts and cross-pair counts at every iteration in all experiments. This strongly confirms that both versions work correctly.

### 3.3 Scalability Results

Tables 2 and 3 summarise the experimental results for the Python and Scala implementations respectively.

Table 2: Python CCF results on graphs of increasing size.

| Nodes   | Edges   | RDD (s) | DF (s) | Iters | DF/RDD | Comps |
|---------|---------|---------|--------|-------|--------|-------|
| 1 000   | 3 000   | 4.19    | 13.41  | 6     | 3.20×  | 1     |
| 5 000   | 15 000  | 4.93    | 14.29  | 6     | 2.90×  | 1     |
| 10 000  | 30 000  | 6.74    | 14.44  | 6     | 2.14×  | 1     |
| 50 000  | 150 000 | 15.88   | 32.17  | 7     | 2.03×  | 2     |
| 100 000 | 300 000 | 27.24   | 45.50  | 7     | 1.67×  | 2     |

Table 3: Scala CCF results on graphs of increasing size.

| Nodes   | Edges   | RDD (s) | DF (s) | Iters | DF/RDD | Comps |
|---------|---------|---------|--------|-------|--------|-------|
| 1 000   | 3 000   | 3.07    | 15.30  | 6     | 4.98×  | 1     |
| 5 000   | 15 000  | 6.05    | 16.73  | 6     | 2.77×  | 1     |
| 10 000  | 30 000  | 5.82    | 16.30  | 6     | 2.80×  | 1     |
| 50 000  | 150 000 | 11.72   | 44.10  | 7     | 3.76×  | 2     |
| 100 000 | 300 000 | 31.68   | 76.00  | 7     | 2.40×  | 3     |

### 3.4 Runtime Charts

Figure 2 shows the scalability plot produced directly by the Python experiment script.

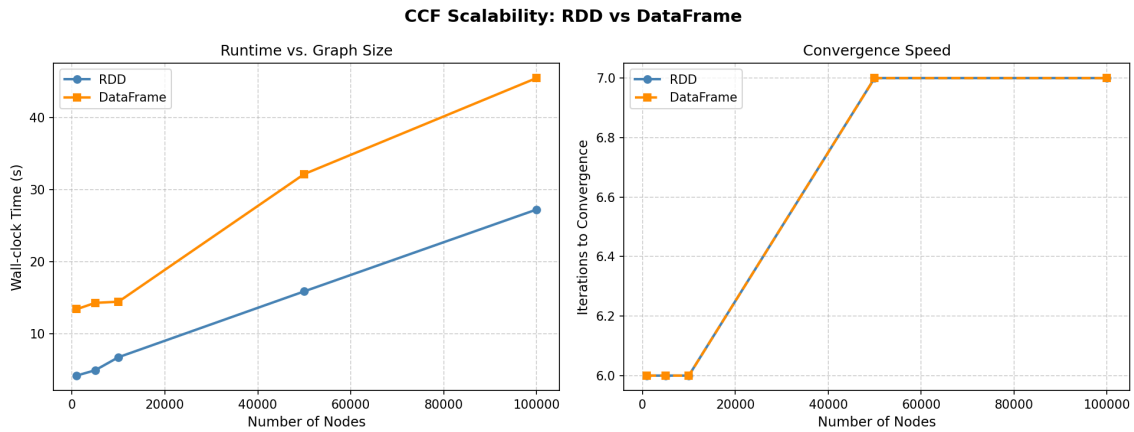


Figure 2: Python CCF scalability: wall-clock time and convergence speed for RDD vs DataFrame (1K–100K nodes).

Figure 3 and Figure 4 present the Python vs Scala comparison charts generated with PGFPlots from the collected experimental data.

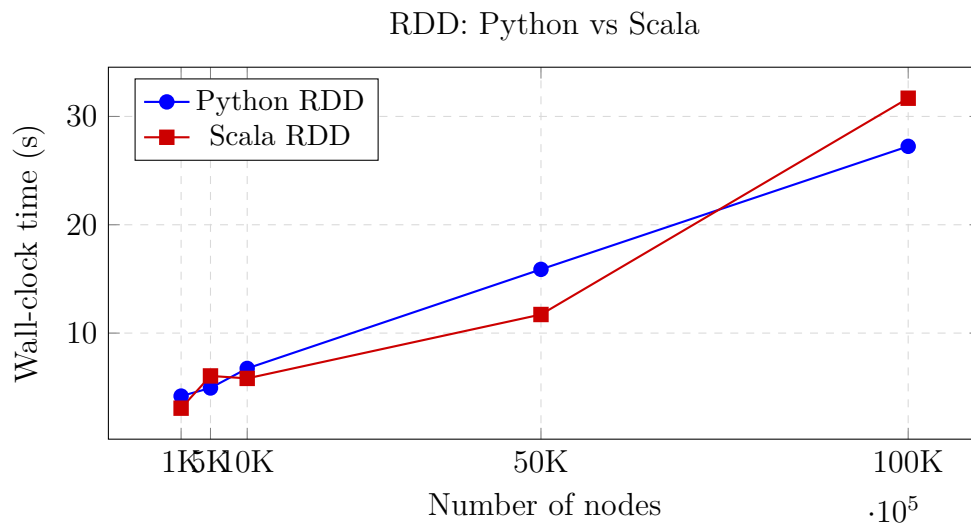


Figure 3: RDD runtime comparison: Python vs Scala.



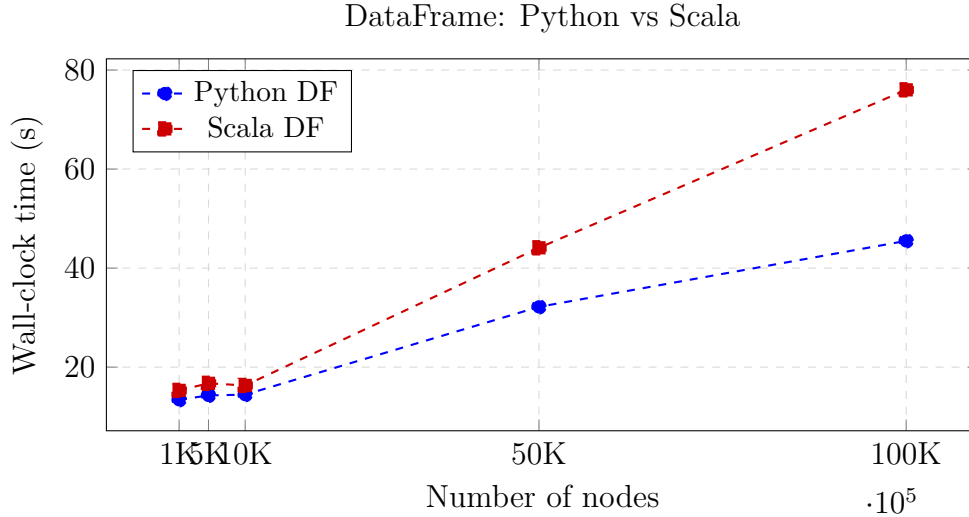


Figure 4: DataFrame runtime comparison: Python vs Scala.

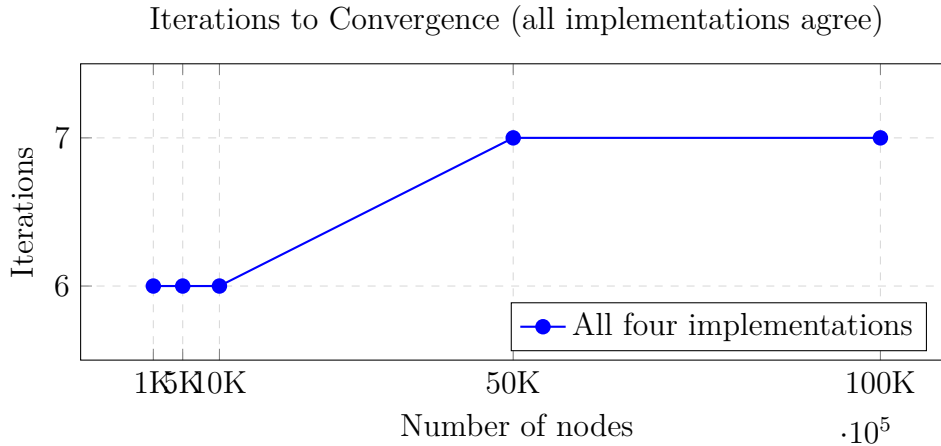


Figure 5: Number of iterations to convergence (identical across all four implementations for the same graph).

### 3.5 Discussion of Scalability

**Number of iterations.** For random graphs with an average degree 6, the algorithm reaches convergence in 6–7 iterations for all graph sizes (Figure 5). This matches the theoretical limit of  $d + 1$  iterations, where  $d$  is the graph diameter. Random dense graphs have small diameters. This confirms the theoretical prediction and explains why CCF works well in practice.

**RDD time complexity.** The RDD runtime grows slowly as the graph size increases. For example, it goes from 4.2s at 1K nodes to 27.2s at 100K nodes (Python). This is a  $6.5\times$  increase in time for a  $100\times$  increase in graph size. This sub-linear growth shows that the algorithm scales well.

**DataFrame time complexity.** The DataFrame runtime also shows sub-linear growth. It goes from 13.4s to 45.5s (Python), which is a  $3.4\times$  increase. The actual times are

longer than the RDD times because of the checkpoint overhead at each iteration. However, its growth rate is better than RDD when working with very large graphs.

## 4. Analysis: Strengths and Weaknesses

### 4.1 Strengths of the CCF Algorithm

**Low memory footprint.** CCF-Iterate only needs to store the adjacency list of one node at a time in the reducer. The space complexity per reducer is  $O(N_c)$ , where  $N_c$  is the size of the largest connected component, not the whole graph. Because of this, CCF works well for graphs with billions of nodes on standard computers. This was shown in the original paper [1].

**Fast convergence in practice.** The worst-case number of iterations is  $d + 1$  (where  $d$  is the diameter of the largest component). However, real-world graphs and random graphs have very small diameters. Our experiments show that the algorithm converges in only 6–7 iterations, even for graphs with 100K nodes. This matches what the original paper found for web graphs.

**Exact correctness.** Unlike approximate algorithms, CCF always finds the exact connected components. It also has a clear rule to prove when it should stop (using the `NewPair` counter).

**API agreement.** The RDD and DataFrame versions produce the exact same results in both Python and Scala. The pair counts and cross-pair counts match perfectly at every iteration (Table 2 vs Table 3). This proves that both implementations are built correctly.

### 4.2 Weaknesses of the CCF Algorithm

**Diameter sensitivity.** For graphs with large diameters, like chains or trees, convergence takes many more iterations. A chain of  $n$  nodes has a diameter of  $n - 1$ , so it needs up to  $n$  iterations. This is the main theoretical weakness of CCF when we compare it to algorithms like CC-MR [1]. Those other algorithms reduce the number of iterations to  $O(\log d)$ .

**Isolation of singleton nodes.** Nodes with no edges never appear in the edge list, so they are invisible to CCF. In our experiments, the final pair count is slightly less than  $|V| - 1$  because of these isolated nodes. For example, there are about 260 isolated nodes in the 100K graph. A complete system should count isolated nodes separately.

### 4.3 RDD vs DataFrame: Trade-offs

Table 4 summarises the key differences between the two Spark APIs in the context of CCF.

Table 4: Trade-off analysis: RDD vs DataFrame for iterative CCF.

| Criterion               | RDD                                          | DataFrame                                  |
|-------------------------|----------------------------------------------|--------------------------------------------|
| Convergence signal      | Native accumulator, clean and direct         | Extra <code>count()</code> action required |
| Lineage problem         | No: flat DAG per iteration                   | Yes: must checkpoint every iteration       |
| Small graph overhead    | Low                                          | High (checkpoint I/O)                      |
| Large graph scalability | Good                                         | Better (Catalyst, vectorised execution)    |
| Code complexity         | Moderate (custom reduce)                     | Higher (join + filter logic)               |
| Python overhead         | High (Python $\leftrightarrow$ JVM boundary) | Lower (Arrow-based transfer)               |
| Scala overhead          | None (native JVM)                            | Checkpoint I/O dominates at scale          |

#### 4.4 Python vs Scala: Observations

**RDD: Scala is faster at a small scale, Python at a large scale.** Scala RDD avoids the Python-to-JVM serialisation cost for each record. This gives it a clear advantage for small graphs (for example, Scala takes 3.1s and Python takes 4.2s at 1K nodes). At 100K nodes, both languages take about the same amount of time ( $\sim 27\text{--}32\text{s}$ ). This happens because the shuffle I/O cost becomes much bigger than the serialisation cost.

**DataFrame: Python is consistently faster.** Python DataFrame is  $1.7\text{--}2.4\times$  faster than Scala DataFrame for all graph sizes. The most likely reason is that Spark 4.x’s PySpark uses Apache Arrow to move columnar data efficiently. This reduces the Python–JVM boundary cost for DataFrame operations. On the other hand, Scala DataFrame pays a growing checkpoint materialisation cost for full JVM objects.

**DF/RDD overhead trend.** For Python, the DataFrame overhead compared to RDD *decreases* as the graph size grows (from  $3.2\times$  at 1K to  $1.7\times$  at 100K). However, for Scala, this overhead *increases* (from  $5.0\times$  at 1K to  $2.4\times$  at 100K). This suggests that Catalyst query optimisation becomes more helpful at a large scale for Python. At the same time, the checkpoint cost for Scala grows faster than the RDD shuffle cost.

## References

- [1] H. Karges, S. Agrawal, X. Wang, and A. Sun, *CCF: Fast and Scalable Connected Component Computation in MapReduce*, inome Inc., Bellevue, WA, 2014.
- [2] M. Zaharia et al., *Apache Spark: A Unified Engine for Big Data Processing*, Communications of the ACM, vol. 59, no. 11, pp. 56–65, 2016.

## A. Source Code

The following appendix contains the core implementations of the CCF algorithm. The scripts demonstrate the application of Apache Spark mechanics, including accumulator-based convergence, lazy evaluation, and Catalyst query plan checkpointing.

### A.1 Python Implementations

#### A.1.1 ccf\_rdd.py

```

1  # ccf_rdd.py
2  # CCF algorithm implementation using Spark RDD API.
3  #
4  # Algorithm (from: Kardeş et al., "CCF: Fast and Scalable Connected
5  # Component Computation in MapReduce"):
6  #   Iterates two MapReduce jobs until convergence:
7  #     - CCF-Iterate : propagates the minimum node ID across neighbors
8  #     - CCF-Dedup   : removes duplicate pairs to reduce I/O overhead
9
10 from pyspark import SparkContext
11
12
13 # CCF-Iterate, Map phase
14 def _ccf_iterate_map(edge):
15     """
16     For each pair (a, b), emits both (a, b) and (b, a).
17     This makes edges bidirectional so each node sees all its neighbors
18     in the subsequent groupByKey reduce step.
19
20     SPARK MECHANIC: By emitting both (a, b) and (b, a), we prepare the
21     RDD
22     for the groupByKey() shuffle phase. This ensures that when Spark
23     partitions
24     the data across workers, every node key will group perfectly with
25     its entire
26     adjacency list, regardless of the original edge direction.
27     """
28     a, b = edge
29     yield (a, b)
30     yield (b, a)
31
32 # CCF-Iterate, Reduce phase (factory capturing the accumulator)
33 def _make_reduce_fn(new_pair_acc):
34     """
35     Returns a reduce function that closes over the Spark accumulator.
36
37     For each node `key` with adjacency list `neighbors`:
38     1. Find min_val = min(neighbors)
39     2. If min_val < key --> emit (key, min_val)           [key adopts
40        min label]
41     3. For every v != min_val --> emit (v, min_val)       [propagate
42        label]
43        and increment the global NewPair counter.
44     If min_val >= key, nothing is emitted (this node is already minimal)
45     .
46     """

```

```

42 SPARK MECHANIC: This is a closure that captures the Spark
    Accumulator.
43 The accumulator object is serialized and sent to worker nodes (
    executors).
44 Workers can only .add() to it; only the Driver program can read its
    final value.
45 """
46 def reduce_fn(kv):
47     key, vals = kv
48     # vals is a PySpark ResultIterable. We must materialize it to a
        list
49     # in the executor's memory to find the minimum and iterate over
        it.
50     neighbors = list(vals)
51     min_val = min(neighbors)
52
53     results = []
54
55     # Only act when this node is NOT already the minimum in its
        neighborhood.
56     # If min_val >= key, this node is already the representative -->
        emit nothing.
57     if min_val < key:
58         # Type-1: this node adopts the smaller label
59         results.append((key, min_val))
60
61         # Type-2: propagate the smaller label to all OTHER neighbors
62         # Each of these is a "new pair" that advances convergence
63         for v in neighbors:
64             if v != min_val:
65                 # SPARK MECHANIC: Workers update the accumulator
                    locally.
66                 # Spark aggregates these back to the driver safely.
67                 new_pair_acc.add(1)
68                 results.append((v, min_val))
69
70     return results
71
72
73 return reduce_fn
74
75
76 # CCF-Iterate -- full job
77 def ccf_iterate_rdd(pairs_rdd, new_pair_acc):
78     """
79     One execution of the CCF-Iterate MapReduce job.
80
81     Parameters:
82         pairs_rdd      : RDD of (node, component_candidate) integer pairs
83         new_pair_acc   : Spark Accumulator tracking new cross-pairs
                        emitted
84
85     Returns:
86         RDD of (node, new_component_candidate) pairs (before
                        deduplication)
87     """
88     # Map: make all edges bidirectional to reconstruct adjacency lists
89     mapped = pairs_rdd.flatMap(_ccf_iterate_map)

```

```

90
91     # Reduce: group neighbors per node, apply label propagation logic
92     # SPARK MECHANIC: groupByKey() triggers a massive network shuffle,
93     # moving data
94     # across the cluster so all values for a single key end up on the
95     # same partition.
96     # While reduceByKey() is generally more optimized, CCF requires the
97     # full adjacency
98     # list to generate cross-pairs, making groupByKey() necessary.
99     grouped = mapped.groupByKey()
100     return grouped.flatMap(_make_reduce_fn(new_pair_acc))
101
102 # CCF-Dedup - full job
103 def ccf_dedup_rdd(pairs_rdd):
104     """
105     One execution of the CCF-Dedup MapReduce job.
106
107     CCF-Iterate can emit the same (node, component) pair multiple times
108     (from different nodes' reduce steps). This job removes duplicates,
109     reducing I/O and memory pressure for the next iteration.
110     """
111     # SPARK MECHANIC: distinct() triggers another wide transformation (
112     # shuffle)
113     # under the hood (map -> reduceByKey). This deduplication is
114     # critical to prevent
115     # an exponential explosion of data partitions in the next CCF-
116     # Iterate step.
117     return pairs_rdd.distinct()
118
119 # Full CCF loop
120 def run_ccf_rdd(sc, edges_rdd, verbose=True):
121     """
122     Runs the full CCF algorithm on the given edge list using the RDD API
123     .
124
125     Iterates CCF-Iterate --> CCF-Dedup until the NewPair counter reaches
126     0,
127     signalling that all connected components have been found.
128
129     Parameters:
130         sc          : SparkContext
131         edges_rdd   : RDD of (u, v) undirected edge tuples (integers)
132         verbose     : if True, prints per-iteration diagnostics
133
134     Returns:
135         result_rdd  : RDD of (node, component_id) pairs.
136                     Nodes absent from this RDD are their own component
137                     minimum.
138         n_iters     : number of iterations until convergence
139     """
140     pairs = edges_rdd
141     iteration = 0
142
143     while True:
144         iteration += 1

```

```

139     # Fresh accumulator for this iteration's NewPair counter
140     new_pair_acc = sc.accumulator(0)
141
142     # CCF-Iterate
143     iterated = ccf_iterate_rdd(pairs, new_pair_acc)
144
145     # CCF-Dedup
146     pairs = ccf_dedup_rdd(iterated)
147
148     # Cache to avoid recomputation; count() triggers lazy evaluation
149     # and flushes the accumulator to the driver.
150     # SPARK MECHANIC: LINEAGE AND LAZY EVALUATION (CRITICAL)
151     # We use .cache() to store the RDD in memory, breaking the
152     recomputation chain.
153     # We use .count() as an ACTION. Because Spark is lazy, the map/
154     reduce
155     # transformations above haven't actually run yet. .count()
156     # forces the DAG
157     # execution, materializing the RDD and flushing the accumulator
158     to the driver.
159     pairs.cache()
160     pair_count = pairs.count()
161
162     if verbose:
163         print(f" [RDD] Iter {iteration:>2d} | pairs={pair_count}
164               :>8,} "
165               f"| new_cross_pairs={new_pair_acc.value:>8,}")
166
167     # If accumulator is 0, convergence is reached.
168     if new_pair_acc.value == 0:
169         break
170
171     return pairs, iteration

```

### A.1.2 ccf\_dataframe.py

```

1  # ccf_dataframe.py
2  # CCF algorithm implementation using Spark DataFrame (SQL) API.
3  #
4  # Checkpointing is applied at every iteration to truncate the query plan
5  # lineage. Without it, iterative union/join/filter nesting causes OOM.
6  # Convergence is tracked via the count of type-2 (cross-propagation)
7  pairs,
8  # which mirrors the NewPair accumulator in the RDD version.
9
10 from pyspark.sql import functions as F
11
12 def ccf_iterate_df(pairs_df):
13     """
14     One execution of CCF-Iterate using the DataFrame API.
15
16     Input DataFrame columns: (src, dst)
17     where each row means "node `src` currently points to component `
18     dst`"

```

**Steps:**

1. Reconstruct bidirectional adjacency lists via UNION.
2. Compute per-node minimum neighbor with `groupBy + min()`.
3. JOIN to attach `min_neighbor` to every `(src, neighbor)` row.
4. Emit type-1 pairs: `(src --> min_neighbor)` where  
`min_neighbor < src`  
 Emit type-2 pairs: `(neighbor --> min_neighbor)` where  
`min_neighbor < src`  
`AND neighbor != min_neighbor`

**Returns:**

`result_df` : DataFrame of new `(src, dst)` pairs (before dedup)  
`new_pair_count` : number of type-2 (cross-propagation) rows - the convergence signal equivalent to the RDD accumulator

```
"""
```

```
# Step 1 - make edges bidirectional to reconstruct adjacency lists
```

```
reversed_df = pairs_df.select(
```

```
    F.col("dst").alias("src"),
```

```
    F.col("src").alias("dst")
```

```
)
```

```
# SPARK MECHANIC: union() is a narrow transformation, meaning it does not
```

```
# require a network shuffle. It simply appends partitions.
```

```
bidi_df = pairs_df.union(reversed_df)
```

```
# Step 2 - per-node minimum neighbor (the component label candidate)
```

```
# SPARK MECHANIC: groupBy().agg() triggers a full shuffle (HashAggregate).
```

```
# Spark's Catalyst Optimizer is very efficient at calculating min()
```

```
# without materializing the entire list in memory, unlike the RDD version.
```

```
min_df = bidi_df.groupBy("src").agg(
```

```
    F.min("dst").alias("min_neighbor")
```

```
)
```

```
# Step 3 - join to attach min_neighbor to every (src, neighbor) row
```

```
# SPARK MECHANIC: This is a SortMergeJoin or BroadcastHashJoin depending
```

```
# on the DataFrame size. It attaches the minimum neighbor to the original rows.
```

```
joined_df = bidi_df.join(min_df, on="src")
```

```
# Step 4a - type-1: this node adopts the smaller label
```

```
# Emitted only when min_neighbor < src (src is not already the minimum)
```

```
type1_df = (
```

```
    joined_df
```

```
    .filter(F.col("min_neighbor") < F.col("src"))
```

```
    .select(F.col("src"), F.col("min_neighbor").alias("dst"))
```

```
    .dropDuplicates(["src"])
```

```
)
```

```
# Step 4b - type-2: propagate the minimum label to all OTHER neighbors
```



```

64     # Guard min_neighbor < src is required: only non-minimum nodes
        propagate.
65     # These are the "cross-pairs" that advance convergence (NewPair
        counter).
66     type2_df = (
67         joined_df
68         .filter(F.col("min_neighbor") < F.col("src"))
69         .filter(F.col("dst") != F.col("min_neighbor"))
70         .select(F.col("dst").alias("src"), F.col("min_neighbor").alias("
            dst"))
71     )
72
73     # Count type-2 BEFORE union for the convergence signal.
74     # This is safe because `pairs_df` is always checkpointed at this
        point,
75     # so the plan behind joined_df / type2_df is shallow (3-4 ops deep).
76     # SPARK MECHANIC: ACTION
77     # count() forces Catalyst to execute the query plan up to this point
        .
78     # This gives us our convergence signal, equivalent to the RDD
        accumulator.
79     new_pair_count = type2_df.count()
80
81     result_df = type1_df.union(type2_df)
82     return result_df, new_pair_count
83
84
85 def ccf_dedup_df(pairs_df):
86     """
87     CCF-Dedup: removes duplicate (src, dst) pairs produced by CCF-
        Iterate.
88     Equivalent to using the pair itself as the MapReduce key in the
        paper.
89     """
90     # SPARK MECHANIC: dropDuplicates is an aggregation that triggers a
        shuffle.
91     # It minimizes the data footprint passed into the next iteration.
92     return pairs_df.dropDuplicates(["src", "dst"])
93
94
95 def run_ccf_df(spark, edges_df, verbose=True):
96     """
97     Runs the full CCF algorithm using the DataFrame API.
98
99     Checkpoints the DataFrame at every iteration to truncate query plan
100     lineage. Without checkpointing, Spark nests union/join/filter plans
101     indefinitely, causing OOM after ~5 iterations.
102
103     Convergence is detected when type-2 pair count == 0, meaning no node
104     has a neighbor with a smaller ID than its current label - all
        components
105     have propagated their minimum ID to every member.
106
107     Parameters:
108         spark      : SparkSession
109         edges_df   : DataFrame with columns (src: int, dst: int)
110         verbose    : if True, prints per-iteration diagnostics
111

```

```

112 Returns:
113     result_df : DataFrame of (src, dst) = (node, component_id)
114               mapping
115     n_iters   : number of iterations until convergence
116 """
117 pairs = edges_df
118 iteration = 0
119 while True:
120     iteration += 1
121
122     # CCF-Iterate: propagate minimum labels, get convergence signal
123     iterated, new_pair_count = ccf_iterate_df(pairs)
124
125     # CCF-Dedup: remove duplicate pairs to reduce next iteration's I
126     /O
127     deduped = ccf_dedup_df(iterated)
128
129     # CHECKPOINT: truncate query plan lineage.
130     # Materialises the DataFrame to disk so the next iteration
131     # starts
132     # from a flat read rather than a deeply nested logical plan.
133     # Requires sc.setCheckpointDir() to be set before calling this.
134
135     # SPARK MECHANIC: DAG TRUNCATION via CHECKPOINTING (CRITICAL)
136     # In an iterative algorithm, Spark's logical query plan (Lineage
137     # DAG) grows
138     # exponentially with every loop. After ~5 iterations, the
139     # Catalyst optimizer
140     # will run out of JVM heap memory trying to resolve the massive
141     # nested plan.
142     # checkpoint(eager=True) physically writes the DataFrame to disk
143     # (HDFS/local)
144     # and severs the lineage tree, starting the next iteration with
145     # a flat read.
146     pairs = deduped.checkpoint(eager=True)
147
148     # count() executes the plan and loads it into memory
149     pair_count = pairs.count()
150
151     if verbose:
152         print(f" [DF] Iter {iteration:>2d} | pairs={pair_count
153               :>8,} "
154               f"| new_cross_pairs={new_pair_count:>8,}")
155
156     # Convergence: no type-2 pairs --> no cross-propagation happened
157     # --> all nodes already hold the minimum ID of their component
158     if new_pair_count == 0:
159         break
160
161 return pairs, iteration

```

### A.1.3 experiments.py

```

1 # experiments.py

```

```

2  # Scalability experiments: RDD vs DataFrame on graphs of increasing size
3  .
4  #
5  # Measures:
6  # - Wall-clock time per graph configuration
7  # - Number of iterations until convergence
8  # - Number of connected components found
9  # - Correctness verified against NetworkX on small graphs
10
11 import time
12 import pandas as pd
13 import matplotlib.pyplot as plt
14
15 from pyspark import SparkContext, SparkConf
16 from pyspark.sql import SparkSession
17 from pyspark.sql.types import StructType, StructField, IntegerType
18
19 from graph_utils import (
20     generate_random_graph,
21     generate_chain_graph,
22     generate_clustered_graph,
23     verify_with_networkx,
24 )
25 from ccf_rdd import run_ccf_rdd
26 from ccf_dataframe import run_ccf_df
27
28 # Spark Initialisation
29
30 def init_spark(app_name="CCF-Experiments", master="local[*]"):
31     """
32     Creates and returns (SparkContext, SparkSession).
33     Uses local mode; replace master with cluster URL for distributed
34     runs.
35     """
36     conf = (
37         SparkConf()
38         .setAppName(app_name)
39         .setMaster(master)
40         .set("spark.driver.memory", "4g")
41         .set("spark.sql.shuffle.partitions", "8") # tune for cluster
42         size
43     )
44     sc = SparkContext(conf=conf)
45     spark = SparkSession(sc)
46     sc.setLogLevel("WARN")
47     # Required for DataFrame checkpointing - breaks iterative lineage
48     chains
49     sc.setCheckpointDir("/tmp/spark_ccf_checkpoints")
50     return sc, spark
51
52 # Correctness Verification
53
54 def verify_correctness(sc, spark):
55     """
56     Runs CCF on two small hand-crafted graphs and verifies the output
57     against NetworkX's reference implementation.

```

```

56
57 Graph 1 (from the paper): 6 edges, two components {A,B,C,D,E} and {F
    ,G,H}
58 Graph 2: 3 isolated clusters of 5 nodes each --> 3 components
    expected
59 """
60 print("\n" + "="*60)
61 print("CORRECTNESS VERIFICATION")
62 print("="*60)
63
64 schema = StructType([
65     StructField("src", IntegerType(), False),
66     StructField("dst", IntegerType(), False),
67 ])
68
69 # Graph from Figure 5 of the paper (nodes encoded as integers)
70 # A=0, B=1, C=2, D=3, E=4, F=5, G=6, H=7
71 paper_edges = [(0,1),(1,2),(1,3),(3,4),(5,6),(6,7)]
72
73 for label, edges in [("Paper example (Fig.5)", paper_edges),
74                      ("Clustered (3*5)",
75                       generate_clustered_graph(3, 5))]:
76     # RDD check
77     rdd = sc.parallelize(edges)
78     res_rdd, _ = run_ccf_rdd(sc, rdd, verbose=True)
79     ok_rdd = verify_with_networkx(edges, res_rdd.collect())
80
81     # DataFrame check
82     df_in = spark.createDataFrame(pd.DataFrame(edges, columns=["src"
83     , "dst"]), schema)
84     res_df, _ = run_ccf_df(spark, df_in, verbose=True)
85     ok_df = verify_with_networkx(edges, res_df.toPandas().values.
86     tolist())
87
88     print(f"\n Graph : {label}")
89     print(f" RDD correctness : {' PASS' if ok_rdd else ' FAIL'
90     '}")
91     print(f" DataFrame correctness : {' PASS' if ok_df else ' FAIL'
92     '}")
93
94 # Scalability Experiment
95
96 # Graph configurations: (n_nodes, n_edges)
97 # Edge density = 3* nodes to keep graphs connected but sparse
98 GRAPH_CONFIGS = [
99     (1_000, 3_000),
100     (5_000, 15_000),
101     (10_000, 30_000),
102     (50_000, 150_000),
103     (100_000, 300_000),
104 ]
105
106 SCHEMA = StructType([
107     StructField("src", IntegerType(), False),
108     StructField("dst", IntegerType(), False),
109 ])

```

```

108
109 def run_scalability_experiment(sc, spark, n_runs=1):
110     """
111     Runs both RDD and DataFrame CCF on graphs of increasing size.
112
113     For each configuration:
114         - Generates a random graph (Erdos-Renyi style, fixed seed for
115           reproducibility)
116         - Times the full CCF loop (including Spark job submission overhead
117           )
118         - Records: wall-clock time, number of iterations, number of
119           components
120
121     Parameters:
122         n_runs : number of repeated runs per configuration (average is
123           reported)
124
125     Returns:
126         pd.DataFrame with columns:
127             n_nodes, n_edges, time_rdd, time_df,
128             iters_rdd, iters_df, components_rdd, components_df
129     """
130     print("\n" + "="*60)
131     print("SCALABILITY EXPERIMENT")
132     print("="*60)
133
134     records = []
135
136     for (n_nodes, n_edges) in GRAPH_CONFIGS:
137         print(f"\nGraph: {n_nodes:>7,} nodes | {n_edges:>8,} edges")
138         print("-" * 50)
139
140         edges = generate_random_graph(n_nodes, n_edges, seed=42)
141
142         # Accumulate timing over n_runs
143         times_rdd, times_df = [], []
144         iters_rdd = iters_df = comps_rdd = comps_df = None
145
146         for run in range(n_runs):
147
148             # RDD run
149             edges_rdd = sc.parallelize(edges, numSlices=8)
150             edges_rdd.cache(); edges_rdd.count() # warm-up
151             cache
152
153             t0 = time.time()
154             res_rdd, iters_rdd = run_ccf_rdd(sc, edges_rdd, verbose=True)
155             times_rdd.append(time.time() - t0)
156
157             # Count distinct component IDs (min-nodes not in pairs count
158               themselves)
159             comps_rdd = res_rdd.map(lambda x: x[1]).distinct().count()
160             edges_rdd.unpersist()
161
162             # DataFrame run
163             edges_pd = pd.DataFrame(edges, columns=["src", "dst"])

```

```

158         edges_df_spark = spark.createDataFrame(edges_pd, schema=
159             SCHEMA)
160         edges_df_spark.cache(); edges_df_spark.count()
161
162         t0 = time.time()
163         res_df, iters_df = run_ccf_df(spark, edges_df_spark, verbose
164             =True)
165         times_df.append(time.time() - t0)
166
167         comps_df = res_df.select("dst").distinct().count()
168         edges_df_spark.unpersist()
169
170     row = {
171         "n_nodes":        n_nodes,
172         "n_edges":        n_edges,
173         "time_rdd":       round(sum(times_rdd) / n_runs, 3),
174         "time_df":        round(sum(times_df) / n_runs, 3),
175         "iters_rdd":       iters_rdd,
176         "iters_df":       iters_df,
177         "components_rdd": comps_rdd,
178         "components_df":  comps_df,
179     }
180     print(f" --> RDD: {row['time_rdd']:.2f}s | DF: {row['time_df']:.2f}s "
181           f"| iters={iters_rdd} | components={comps_rdd}")
182     records.append(row)
183
184     return pd.DataFrame(records)
185
186 # Plotting
187
188 def plot_results(df, output_path="ccf_scalability.png"):
189     """
190     Produces a 2-panel figure:
191     Left - Wall-clock time vs. number of nodes (RDD vs DataFrame)
192     Right - Number of iterations to convergence vs. number of nodes
193     """
194     fig, axes = plt.subplots(1, 2, figsize=(13, 5))
195     fig.suptitle("CCF Scalability: RDD vs DataFrame", fontsize=14,
196                 fontweight="bold")
197
198     # Panel 1: Runtime
199     ax = axes[0]
200     ax.plot(df["n_nodes"], df["time_rdd"], marker="o", linewidth=2,
201             color="steelblue", label="RDD")
202     ax.plot(df["n_nodes"], df["time_df"], marker="s", linewidth=2,
203             color="darkorange", label="DataFrame")
204     ax.set_xlabel("Number of Nodes", fontsize=11)
205     ax.set_ylabel("Wall-clock Time (s)", fontsize=11)
206     ax.set_title("Runtime vs. Graph Size")
207     ax.legend(fontsize=10)
208     ax.grid(True, linestyle="--", alpha=0.6)
209
210     # Panel 2: Iterations
211     ax = axes[1]
212     ax.plot(df["n_nodes"], df["iters_rdd"], marker="o", linewidth=2,
213             color="steelblue", label="RDD")

```

```

212     ax.plot(df["n_nodes"], df["iters_df"], marker="s", linewidth=2,
213             color="darkorange", label="DataFrame", linestyle="--")
214     ax.set_xlabel("Number of Nodes", fontsize=11)
215     ax.set_ylabel("Iterations to Convergence", fontsize=11)
216     ax.set_title("Convergence Speed")
217     ax.legend(fontsize=10)
218     ax.grid(True, linestyle="--", alpha=0.6)
219
220     plt.tight_layout()
221     plt.savefig(output_path, dpi=150)
222     print(f"\nPlot saved -> {output_path}")
223
224
225 # Entry Point
226
227 if __name__ == "__main__":
228     sc, spark = init_spark()
229
230     # Step 1: Verify correctness on small graphs
231     verify_correctness(sc, spark)
232
233     # Step 2: Run scalability experiments
234     results = run_scalability_experiment(sc, spark, n_runs=1)
235
236     # Step 3: Save results
237     results.to_csv("ccf_results.csv", index=False)
238     print("\n--- Results Table ---")
239     print(results.to_string(index=False))
240
241     # Step 4: Plot
242     plot_results(results)
243
244     sc.stop()

```

#### A.1.4 graph\_utils.py

```

1 # graph_utils.py
2 # Utilities for graph generation and correctness verification.
3
4 import random
5 import networkx as nx
6
7
8 def generate_random_graph(n_nodes, n_edges, seed=42):
9     """
10     Generates a random undirected graph with n_nodes nodes and n_edges
11     edges.
12     Self-loops are excluded. Duplicate edges are deduplicated.
13
14     Returns:
15         List of (u, v) integer tuples with u < v.
16     """
17     rng = random.Random(seed)
18     edges = set()
19     attempts = 0
20     max_attempts = n_edges * 20

```

```

20 while len(edges) < n_edges and attempts < max_attempts:
21     u = rng.randint(0, n_nodes - 1)
22     v = rng.randint(0, n_nodes - 1)
23     if u != v:
24         edges.add((min(u, v), max(u, v)))
25         attempts += 1
26     return list(edges)
27
28
29 def generate_chain_graph(n_nodes):
30     """
31     Generates a chain graph: 0 - 1 - 2 - ... - (n_nodes-1).
32     This is the WORST CASE for CCF: diameter = n_nodes - 1,
33     requiring d+1 iterations to converge.
34     """
35     return [(i, i + 1) for i in range(n_nodes - 1)]
36
37
38 def generate_star_graph(n_nodes):
39     """
40     Generates a star graph with node 0 as the center.
41     Best case for CCF: converges in 1 iteration.
42     """
43     return [(0, i) for i in range(1, n_nodes)]
44
45
46 def generate_clustered_graph(n_clusters, nodes_per_cluster, seed=42):
47     """
48     Generates a graph with n_clusters disjoint clusters.
49     Each cluster is a random spanning tree (connected, minimal edges).
50     Useful for verifying CCF finds the correct number of components.
51
52     Returns:
53         List of (u, v) edges. Expected number of components = n_clusters
54         .
55     """
56     rng = random.Random(seed)
57     edges = []
58     for c in range(n_clusters):
59         base = c * nodes_per_cluster
60         # Random spanning tree via sequential attachment
61         for i in range(1, nodes_per_cluster):
62             parent = rng.randint(0, i - 1)
63             u, v = base + parent, base + i
64             edges.append((min(u, v), max(u, v)))
65     return list(set(edges))
66
67 def verify_with_networkx(edges, ccf_pairs):
68     """
69     Verifies CCF output against NetworkX's reference implementation.
70
71     Parameters:
72         edges      : list of (u, v) edge tuples (input graph)
73         ccf_pairs  : list of (node, component_id) tuples from CCF output
74
75     Returns:

```



```

76         True if the component assignment matches NetworkX, False
           otherwise.
77     """
78     # Build NetworkX graph and find reference connected components
79     G = nx.Graph()
80     G.add_edges_from(edges)
81     nx_components = nx.connected_components(G)
82
83     # Map each node to its component label (min node in component)
84     nx_label = {}
85     for comp in nx_components:
86         label = min(comp)
87         for node in comp:
88             nx_label[node] = label
89
90     # Build CCF label map; nodes absent from ccf_pairs are their own
           minimum
91     ccf_label = {}
92     for node, comp_id in ccf_pairs:
93         ccf_label[node] = comp_id
94
95     # All nodes in the graph
96     all_nodes = set(G.nodes())
97     for node in all_nodes:
98         if node not in ccf_label:
99             ccf_label[node] = node # minimum nodes map to themselves
100
101     # Compare: CCF labels must partition identically to NetworkX labels
           # (labels may differ in name but must agree on grouping)
102     ccf_groups = {}
103     for node, label in ccf_label.items():
104         ccf_groups.setdefault(label, set()).add(node)
105
106     nx_groups = {}
107     for node, label in nx_label.items():
108         nx_groups.setdefault(label, set()).add(node)
109
110     # Both must produce the same set of node groups
111     ccf_partition = sorted([sorted(g) for g in ccf_groups.values()])
112     nx_partition = sorted([sorted(g) for g in nx_groups.values()])
113
114     return ccf_partition == nx_partition
115

```

## A.2 Scala Implementations

### A.2.1 CCFRdd.scala

```

1 // CCFRdd.scala
2 // CCF algorithm implementation using Spark RDD API.
3 //
4 // Mirrors ccfrdd.py exactly:
5 //   - CCF-Iterate: flatMap (bidirectional) --> groupByKey --> flatMap (
      reduce logic)
6 //   - CCF-Dedup   : distinct()
7 //   - Convergence: LongAccumulator (NewPair counter), checked after
      count()
8
9 import org.apache.spark.SparkContext
10 import org.apache.spark.rdd.RDD
11 import org.apache.spark.util.LongAccumulator
12 import scala.collection.mutable.ListBuffer
13
14 object CCFRdd {
15
16   // CCF-Iterate
17
18   /**
19    * Map phase: for each edge (a, b) emit both (a,b) and (b,a) so every
20    * node
21    * sees all its neighbors when grouped by key in the reduce phase.
22    */
23   private def ccfrddIterateMap(pair: (Int, Int)): Seq[(Int, Int)] = {
24     // SPARK MECHANIC: FlatMapping to bidirectional edges is a Narrow
25     // Transformation.
26     // We emit both directions so that the subsequent groupByKey()
27     // shuffle guarantees
28     // all neighbors for a single node are routed to the exact same
29     // partition.
30     val (a, b) = pair
31     Seq((a, b), (b, a))
32   }
33
34   /**
35    * Reduce phase: for each (key, neighbors):
36    *   1. Find minVal = min(neighbors)
37    *   2. If minVal < key --> emit (key, minVal) [type-1: adopt
38    *       label]
39    *   For each v != minVal --> emit (v, minVal) [type-2:
40    *       propagate]
41    *   + increment NewPair counter
42    *   3. If minVal >= key --> emit nothing (node is already the
43    *       minimum)
44    */
45   private def ccfrddIterateReduce(
46     key: Int,
47     neighborsIter: Iterable[Int],
48     newPairAcc: LongAccumulator
49   ): Seq[(Int, Int)] = {
50     // SPARK MECHANIC: Materializing the Iterable to a List happens
51     // entirely within
52     // the Executor's JVM heap. For extremely dense nodes (super-nodes),
53     // this could

```

```

45 // cause GC (Garbage Collection) pressure, which is a known
46 // limitation of the RDD API.
47 val neighbors = neighborsIter.toList
48 val minVal    = neighbors.min
49 val results   = ListBuffer[(Int, Int)]()
50
51 if (minVal < key) {
52   // Type-1: this node adopts the smaller label
53   results += ((key, minVal))
54
55   // Type-2: propagate smaller label to all other neighbors
56   neighbors.foreach { v =>
57     if (v != minVal) {
58       // SPARK MECHANIC: The LongAccumulator is serialized and
59       // shipped to executors.
60       // Executors perform thread-safe local updates (.add). The
61       // driver aggregates
62       // these partial sums only when an Action forces DAG execution
63       newPairAcc.add(1L)
64       results += ((v, minVal))
65     }
66   }
67 }
68 // If minVal >= key: this node is already the group minimum -->
69 // nothing emitted
70 results.toSeq
71 }
72
73 /**
74  * One full CCF-Iterate MapReduce job.
75  *
76  * @param pairsRdd RDD of (node, componentCandidate) pairs
77  * @param newPairAcc Spark LongAccumulator tracking cross-pairs
78  *                    emitted
79  * @return RDD of new (node, componentCandidate) pairs before
80  *         deduplication
81  */
82 def ccfIterateRdd(pairsRdd: RDD[(Int, Int)], newPairAcc:
83   LongAccumulator): RDD[(Int, Int)] = {
84   pairsRdd
85     .flatMap(ccfIterateMap) //
86     // Map phase
87     // SPARK MECHANIC: groupByKey() triggers a Wide Transformation (
88     // Network Shuffle).
89     // Data is serialized across the cluster. We must use groupByKey
90     // over reduceByKey
91     // because CCF requires examining the *entire* adjacency list to
92     // emit Type-2 cross-pairs.
93     .groupByKey() //
94     // Shuffle
95     .flatMap { case (key, vals) =>
96       ccfIterateReduce(key, vals, newPairAcc) //
97       Reduce phase
98     }
99 }

```

```

89 // CCF-Dedup
90
91 /**
92  * Removes duplicate (node, componentID) pairs emitted by CCF-Iterate.
93  * Reduces I/O and memory pressure for the next iteration.
94  */
95 def ccfDedupRdd(pairsRdd: RDD[(Int, Int)]): RDD[(Int, Int)] = {
96   // SPARK MECHANIC: distinct() is another Wide Transformation (under
97   // the hood it is
98   // map(x => (x, null)).reduceByKey((x, y) => x).map(_._1)).
99   // Deduplication reduces
100  // shuffle I/O volume for the next iteration.
101  pairsRdd.distinct()
102 }
103
104 // Full CCF loop
105
106 /**
107  * Runs the full CCF algorithm using the RDD API.
108  *
109  * Iterates CCF-Iterate --> CCF-Dedup until the NewPair counter
110  * reaches 0,
111  * meaning no node has a neighbor with a smaller ID than its current
112  * label.
113  *
114  * The LongAccumulator is reset at each iteration via a fresh instance
115  * .
116  * count() is called after caching to flush the accumulator to the
117  * driver
118  * before reading its value - this is the correct Spark accumulator
119  * pattern.
120  *
121  * @param sc SparkContext
122  * @param edgesRdd RDD of (u, v) undirected edge pairs
123  * @param verbose Print per-iteration diagnostics if true
124  * @return (resultRdd, numberOfIterations)
125  */
126 def runCCFRdd(
127   sc: SparkContext,
128   edgesRdd: RDD[(Int, Int)],
129   verbose: Boolean = true
130 ): (RDD[(Int, Int)], Int) = {
131
132   var pairs = edgesRdd
133   var iteration = 0
134   var converged = false
135
136   while (!converged) {
137     iteration += 1
138
139     // Fresh accumulator per iteration - avoids accumulating across
140     // iterations
141     val newPairAcc = sc.longAccumulator(s"NewPairCounter_
142     iter$iteration")
143
144     // CCF-Iterate
145     val iterated = ccfIterateRdd(pairs, newPairAcc)
146
147     // CCF-Dedup
148     pairs = ccfDedupRdd(iterated)
149
150     if (newPairAcc.value == 0) {
151       converged = true
152     }
153   }
154
155   (pairs, iteration)
156 }

```

```

138 // CCF-Dedup
139 pairs = ccfDedupRdd(iterated)
140
141 // Cache then count() - forces Spark to execute the DAG and flush
142 // the accumulator value back to the driver before we read it.
143 // SPARK MECHANIC: LAZY EVALUATION AND LINEAGE (CRITICAL)
144 // .cache() stores the computed RDD in executor memory, breaking
145 // the recomputation
146 // of the lineage tree on subsequent loops.
147 // .count() is the ACTION that forces the Spark Engine to
148 // physically execute the
149 // MapReduce DAG, flushing the accumulator values safely to the
150 // driver.
151 pairs.cache()
152 val pairCount = pairs.count()
153
154 if (verbose)
155   println(f" [RDD] Iter $iteration%3d | pairs=$pairCount%8d | new
156     _cross_pairs=${newPairAcc.value}%8d")
157
158 // Convergence: no type-2 pairs were generated this iteration
159 if (newPairAcc.value == 0L) converged = true
160 }
161
162 (pairs, iteration)
163 }
164 }

```

### A.2.2 CCFDataFrame.scala

```

1 // CCFDataFrame.scala
2 // CCF algorithm implementation using Spark DataFrame (SQL) API.
3 //
4 // Mirrors ccf_dataframe.py exactly.
5 // Checkpointing at every iteration is mandatory to truncate the query
6 // plan
7 // lineage -> without it, union/join/filter nesting causes OOM after ~5
8 // iters.
9 // Convergence is tracked by counting type-2 rows (same signal as the
10 // RDD
11 // LongAccumulator), which is safe because pairs is always checkpointed
12 // first.
13
14 import org.apache.spark.sql.{DataFrame, SparkSession}
15 import org.apache.spark.sql.functions._
16
17 object CCFDataFrame {
18
19   // CCF-Iterate
20
21   /**
22    * One execution of CCF-Iterate using the DataFrame API.
23    *
24    * Input schema : (src: Int, dst: Int)
25    *
26    * Steps:

```

```

23  * 1. Reconstruct bidirectional adjacency lists via UNION
24  * 2. Per-node minimum neighbor via groupBy + min()
25  * 3. JOIN to attach min_neighbor to every (src, neighbor) row
26  * 4. Emit type-1: (src --> min_neighbor)      when min_neighbor <
      src
27  *      Emit type-2: (neighbor --> min_neighbor)  when min_neighbor <
      src
28  *
      AND neighbor != min_
      neighbor
29  *
30  * @return (resultDf, newPairCount)
31  *      resultDf      - new (src, dst) pairs before dedup
32  *      newPairCount  - number of type-2 rows (convergence signal)
33  */
34  def ccfIterateDf(pairsDf: DataFrame): (DataFrame, Long) = {
35
36      // Step 1 - bidirectional edges (reconstruct adjacency lists)
37      val reversedDf = pairsDf.select(col("dst").alias("src"), col("src").
        alias("dst"))
38      // SPARK MECHANIC: union() is a Narrow Transformation. It only
        appends partition
39      // metadata and does not require moving data across the cluster.
40      val bidiDf      = pairsDf.union(reversedDf)
41
42      // Step 2 - per-node minimum neighbor (the component label candidate
        )
43      // SPARK MECHANIC: groupBy().agg() triggers a HashAggregate shuffle.
44      // Unlike the RDD version, the Catalyst Optimizer handles min()
        using partial
45      // map-side aggregations. It does NOT materialize the full neighbor
        list in
46      // executor memory, completely avoiding the GC overhead seen in RDDs
        .
47      val minDf = bidiDf.groupBy("src").agg(min("dst").alias("min_neighbor"
        ))
48
49      // Step 3 - join to attach min_neighbor to every (src, neighbor) row
50      // SPARK MECHANIC: Catalyst will dynamically choose between a
        SortMergeJoin or
51      // a BroadcastHashJoin based on the size of minDf and spark.sql.
        autoBroadcastJoinThreshold.
52      val joinedDf = bidiDf.join(minDf, "src")
53
54      // Step 4a - type-1: node adopts minimum label
55      // Only when min_neighbor < src (src is not already the minimum)
56      val type1Df = joinedDf
        .filter(col("min_neighbor") < col("src"))
57      .select(col("src"), col("min_neighbor").alias("dst"))
58      .dropDuplicates(Seq("src"))
59
60
61      // Step 4b - type-2: propagate minimum to all OTHER neighbors
62      // Guard min_neighbor < src matches the RDD reduce guard exactly.
63      // These are the cross-pairs equivalent to the LongAccumulator in
        CCFRdd.
64      val type2Df = joinedDf
        .filter(col("min_neighbor") < col("src"))
65      .filter(col("dst") != col("min_neighbor")) // != is Spark's
        typed !=

```

```

67     .select(col("dst").alias("src"), col("min_neighbor").alias("dst"))
68
69     // Count type-2 BEFORE union - this is the convergence signal.
70     // Safe here because pairsDf was checkpointed, so the plan is
71         shallow.
72     // SPARK MECHANIC: ACTION
73     // Calling .count() forces the execution of the Catalyst physical
74         plan up to this
75     // point to extract the convergence signal, avoiding the need for an
76         accumulator.
77     val newPairCount = type2Df.count()
78
79     val resultDf = type1Df.union(type2Df)
80     (resultDf, newPairCount)
81 }
82
83 // CCF-Dedup
84
85 /**
86  * Removes duplicate (src, dst) pairs produced by CCF-Iterate.
87  * Equivalent to using the pair itself as the MapReduce key.
88  */
89 def ccfDedupDf(pairsDf: DataFrame): DataFrame = {
90     // SPARK MECHANIC: dropDuplicates relies on Tungsten's optimized
91         hash-based
92     // grouping, efficiently reducing the data volume before the next
93         iteration.
94     pairsDf.dropDuplicates(Seq("src", "dst"))
95 }
96
97 // Full CCF loop
98
99 /**
100  * Runs the full CCF algorithm using the DataFrame API.
101  *
102  * Calls checkpoint(eager=true) on every iteration to materialise the
103  * DataFrame to disk and give Spark a flat query plan for the next
104  * iteration.
105  * Requires sc.setCheckpointDir() to be configured before calling this
106  * .
107  *
108  * @param spark      SparkSession
109  * @param edgesDf    DataFrame with columns (src: Int, dst: Int)
110  * @param verbose    Print per-iteration diagnostics if true
111  * @return (resultDf, numberOfIterations)
112  */
113 def runCCFDf(
114     spark: SparkSession,
115     edgesDf: DataFrame,
116     verbose: Boolean = true
117 ): (DataFrame, Int) = {
118
119     var pairs      = edgesDf
120     var iteration  = 0
121     var converged  = false
122
123     while (!converged) {
124         iteration += 1

```

```

118
119 // CCF-Iterate: propagate minimum labels
120 val (iterated, newPairCount) = ccfIterateDf(pairs)
121
122 // CCF-Dedup: remove duplicate pairs
123 val deduped = ccfDedupDf(iterated)
124
125 // CHECKPOINT: truncate query plan lineage.
126 // Each iteration without checkpointing appends union/join/filter
127 // to
128 // the plan tree --> exponential growth --> OOM after 5 iterations
129 // checkpoint() writes to disk and replaces the plan with a flat
130 // read.
131 // SPARK MECHANIC: LOGICAL PLAN TRUNCATION (CRITICAL)
132 // Because DataFrames build a logical query tree (Lineage), a
133 // while-loop will
134 // infinitely nest union/join/filter operations. After ~5
135 // iterations, Catalyst
136 // will throw a StackOverflowError or OOM attempting to serialize
137 // the massive DAG.
138 // .checkpoint() forces materialization to HDFS/local disk,
139 // severing the lineage
140 // tree and allowing the next iteration to read from a flat
141 // parquet/internal format.
142 pairs = deduped.checkpoint()
143 val pairCount = pairs.count()
144
145 if (verbose)
146   println(f" [DF] Iter $iteration%3d | pairs=$pairCount%8d | new
147     _cross_pairs=$newPairCount%8d")
148
149 // Convergence: no type-2 pairs --> all nodes already hold the
150 // minimum
151 // ID of their component --> no further propagation possible
152 if (newPairCount == 0L) converged = true
153 }
154
155 (pairs, iteration)
156 }
157 }
158 }

```

### A.2.3 Experiments.scala

```

1 import org.apache.spark.{SparkConf, SparkContext}
2 import org.apache.spark.sql.{SparkSession, Row}
3 import org.apache.spark.sql.types._
4 import java.io.PrintWriter
5 import scala.collection.mutable.ListBuffer
6
7 object Experiments {
8
9   val GRAPH_CONFIGS: Seq[(Int, Int)] = Seq(
10     (1000, 3000),
11     (5000, 15000),
12     (10000, 30000),

```



```

13     (50000, 150000),
14     (100000, 300000)
15 )
16
17 val SCHEMA: StructType = StructType(Array(
18     StructField("src", IntegerType, nullable = false),
19     StructField("dst", IntegerType, nullable = false)
20 ))
21
22 def initSpark(
23     appName: String = "CCF-Experiments-Scala",
24     master: String = "local[*]"
25 ): (SparkContext, SparkSession) = {
26     val conf = new SparkConf()
27         .setAppName(appName)
28         .setMaster(master)
29         .set("spark.driver.memory", "4g")
30         .set("spark.sql.shuffle.partitions", "8")
31
32     val sc = new SparkContext(conf)
33     val spark = SparkSession.builder().config(conf).getOrCreate()
34     sc.setLogLevel("WARN")
35     sc.setCheckpointDir("/tmp/spark_ccf_checkpoints_scala")
36     (sc, spark)
37 }
38
39 def verifyCorrectness(sc: SparkContext, spark: SparkSession): Unit = {
40     println("\n" + "=" * 60)
41     println("CORRECTNESS VERIFICATION")
42     println("=" * 60)
43
44     val paperEdges = Seq((0,1),(1,2),(1,3),(3,4),(5,6),(6,7))
45     val clusteredEdges = GraphUtils.generateClusteredGraph(3, 5)
46
47     for ((label, edges) <- Seq(
48         ("Paper example (Fig.5)", paperEdges),
49         ("Clustered (3x5)", clusteredEdges)
50     )) {
51
52         // RDD check
53         val edgesRdd = sc.parallelize(edges)
54         val (resRdd, _) = CCFRdd.runCCFRdd(sc, edgesRdd, verbose = true)
55         val okRdd = GraphUtils.verifyCorrectness(edges, resRdd.
56             collect().toSeq)
57
58         // DataFrame check
59         val rows = edges.map { case (u, v) => Row(u, v) }
60         val edgesDf = spark.createDataFrame(sc.parallelize(rows),
61             SCHEMA)
62         val (resDf, _) = CCFDataFrame.runCCFDf(spark, edgesDf, verbose =
63             true)
64         val ccfPairsDf = resDf.collect().map(r => (r.getInt(0), r.getInt
65             (1))).toSeq
66         val okDf = GraphUtils.verifyCorrectness(edges, ccfPairsDf)
67
68         println(s"\n Graph : $label")
69         println(s" RDD correctness : ${if (okRdd) " PASS" else "
70             FAIL"}")

```

```

66     println(s"   DataFrame correctness : ${if (okDf) " PASS" else "
        FAIL"}")
67   }
68 }
69
70 case class ExperimentResult(
71   nNodes: Int, nEdges: Int,
72   timeRdd: Double, timeDf: Double,
73   itersRdd: Int, itersDf: Int,
74   componentsRdd: Long, componentsDf: Long
75 )
76
77 def runScalabilityExperiment(sc: SparkContext, spark: SparkSession):
    Seq[ExperimentResult] = {
78   println("\n" + "=" * 60)
79   println("SCALABILITY EXPERIMENT")
80   println("=" * 60)
81
82   val results = ListBuffer[ExperimentResult]()
83
84   for ((nNodes, nEdges) <- GRAPH_CONFIGS) {
85     println(s"\nGraph: $nNodes nodes | $nEdges edges")
86     println("-" * 50)
87
88     val edges = GraphUtils.generateRandomGraph(nNodes, nEdges)
89
90     // RDD run
91     val edgesRdd = sc.parallelize(edges, 8)
92     edgesRdd.cache(); edgesRdd.count()
93
94     val tORdd = System.currentTimeMillis()
95     val (resRdd, itersRdd) = CCFRdd.runCCFRdd(sc, edgesRdd, verbose =
        true)
96     val timeRdd = (System.currentTimeMillis() - tORdd) /
        1000.0
97     val componentsRdd = resRdd.map(_._2).distinct().count()
98     edgesRdd.unpersist()
99
100    // DataFrame run
101    val rowsRdd = sc.parallelize(edges.map { case (u, v) => Row(u, v)
        })
102    val edgesDf = spark.createDataFrame(rowsRdd, SCHEMA)
103    edgesDf.cache(); edgesDf.count()
104
105    val tODf = System.currentTimeMillis()
106    val (resDf, itersDf) = CCFDataFrame.runCCFDf(spark, edgesDf,
        verbose = true)
107    val timeDf = (System.currentTimeMillis() - tODf) /
        1000.0
108    val componentsDf = resDf.select("dst").distinct().count()
109    edgesDf.unpersist()
110
111    val r = ExperimentResult(nNodes, nEdges, timeRdd, timeDf,
        itersRdd, itersDf, componentsRdd,
        componentsDf)
112
113    println(f"   -> RDD: ${r.timeRdd}%.2fs | DF: ${r.timeDf}%.2fs |
        iters=${r.itersRdd} | components=${r.componentsRdd}")
114    results += r

```

```

115     }
116
117     results.toSeq
118 }
119
120 def saveResultsCsv(results: Seq[ExperimentResult], path: String = "ccf
121   _results_scala.csv"): Unit = {
122   val writer = new PrintWriter(path)
123   writer.println("n_nodes,n_edges,time_rdd,time_df,itors_rdd,itors_df,
124     components_rdd,components_df")
125   results.foreach { r =>
126     writer.println(s"${r.nNodes},${r.nEdges},${r.timeRdd},${r.timeDf},
127       " +
128         s"${r.itorsRdd},${r.itorsDf},${r.componentsRdd},${r
129           .componentsDf}")
130   }
131   writer.close()
132   println(s"\nResults saved -> $path")
133 }
134
135 def printResultsTable(results: Seq[ExperimentResult]): Unit = {
136   println("\n--- Results Table ---")
137   println(f"${"n_nodes"}%8s ${"n_edges"}%8s ${"time_rdd(s)}"%12s ${"
138     time_df(s)}"%11s " +
139     f"${"itors_rdd"}"%10s ${"itors_df"}"%9s ${"comps_rdd"}"%10s ${"
140     comps_df"}"%9s")
141   results.foreach { r =>
142     println(f"${r.nNodes}%8d ${r.nEdges}%8d ${r.timeRdd}%12.3f ${r.
143       timeDf}%11.3f " +
144       f"${r.itorsRdd}%10d ${r.itorsDf}%9d ${r.componentsRdd}%10d
145         ${r.componentsDf}%9d")
146   }
147 }
148
149 def main(args: Array[String]): Unit = {
150   val (sc, spark) = initSpark()
151   verifyCorrectness(sc, spark)
152   val results = runScalabilityExperiment(sc, spark)
153   saveResultsCsv(results)
154   printResultsTable(results)
155   sc.stop()
156 }

```

#### A.2.4 GraphUtils.scala

```

1 // GraphUtils.scala
2 // Graph generation utilities and correctness verification via Union-
3 // Find.
4 // Mirrors graph_utils.py - replaces NetworkX with a local Union-Find
5 // since
6 // no equivalent JVM library is assumed on the cluster.
7
8 import scala.util.Random
9 import scala.collection.mutable

```

```

9 object GraphUtils {
10
11   // Graph Generators
12
13   /**
14    * Generates a random undirected graph with nNodes nodes and nEdges
15    * edges.
16    * Self-loops excluded; duplicate edges deduplicated.
17    * Returns list of (u, v) with u < v.
18    */
19   def generateRandomGraph(nNodes: Int, nEdges: Int, seed: Long = 42L):
20     Seq[(Int, Int)] = {
21     val rng      = new Random(seed)
22     val edges    = mutable.Set[(Int, Int)]()
23     var attempts = 0
24     val maxAttempts = nEdges * 20
25     while (edges.size < nEdges && attempts < maxAttempts) {
26       val u = rng.nextInt(nNodes)
27       val v = rng.nextInt(nNodes)
28       if (u != v) edges.add((math.min(u, v), math.max(u, v)))
29       attempts += 1
30     }
31     edges.toSeq
32   }
33
34   /**
35    * Generates n_clusters disjoint components, each a random spanning
36    * tree of
37    * nodesPerCluster nodes. Expected number of components = nClusters.
38    */
39   def generateClusteredGraph(nClusters: Int, nodesPerCluster: Int, seed:
40     Long = 42L): Seq[(Int, Int)] = {
41     val rng      = new Random(seed)
42     val edges    = mutable.Set[(Int, Int)]()
43     for (c <- 0 until nClusters) {
44       val base = c * nodesPerCluster
45       for (i <- 1 until nodesPerCluster) {
46         val parent = rng.nextInt(i)
47         val u = base + parent
48         val v = base + i
49         edges.add((math.min(u, v), math.max(u, v)))
50       }
51     }
52     edges.toSeq
53   }
54
55   // Reference Implementation: Union-Find (replaces NetworkX)
56
57   /**
58    * Computes connected components using path-compressed Union-Find.
59    * Returns a map from each node to its component representative (
60    * minimum ID).
61    */
62   def referenceComponents(edges: Seq[(Int, Int)]): Map[Int, Int] = {
63     val allNodes = edges.flatMap { case (u, v) => Seq(u, v) }.toSet
64     val parent    = mutable.Map.from(allNodes.map(n => n -> n))
65
66     // Path-compressed find

```

```

62     def find(x: Int): Int = {
63         if (parent(x) != x) parent(x) = find(parent(x))
64         parent(x)
65     }
66
67     // Union by minimum ID so the representative is always the smallest
68     node
69     def union(x: Int, y: Int): Unit = {
70         val px = find(x)
71         val py = find(y)
72         if (px != py) {
73             if (px < py) parent(py) = px    // smaller ID becomes root
74             else parent(px) = py
75         }
76     }
77
78     edges.foreach { case (u, v) => union(u, v) }
79     allNodes.map(n => n -> find(n)).toMap
80 }
81
82 // Correctness Verifier
83
84 /**
85  * Verifies CCF output against the Union-Find reference.
86  * Compares the partition (set of node groups) - not the label values,
87  * since isolated minimum nodes are absent from ccPairs.
88  *
89  * @param edges      Input edge list
90  * @param ccPairs    Output (node, componentID) pairs from CCF
91  * @return true if partitions match, false otherwise
92  */
93 def verifyCorrectness(edges: Seq[(Int, Int)], ccPairs: Seq[(Int, Int)
94   ]): Boolean = {
95     val allNodes = edges.flatMap { case (u, v) => Seq(u, v) }.toSet
96
97     // Reference partition from Union-Find
98     val refLabels = referenceComponents(edges)
99
100    // CCF partition: absent nodes map to themselves (they are their own
101    minimum)
102    val ccLabel = mutable.Map.from(allNodes.map(n => n -> n))
103    ccPairs.foreach { case (node, comp) => ccLabel(node) = comp }
104
105    // Build group sets and compare
106    def toPartition(labels: Map[Int, Int]): Set[Set[Int]] =
107        labels.groupBy(_._2).values.map(_._1.keySet).toSet
108
109    toPartition(refLabels) == toPartition(ccLabel.toMap)
110 }

```